

Архитектурный аудит Ventrix / B2B AI Checker

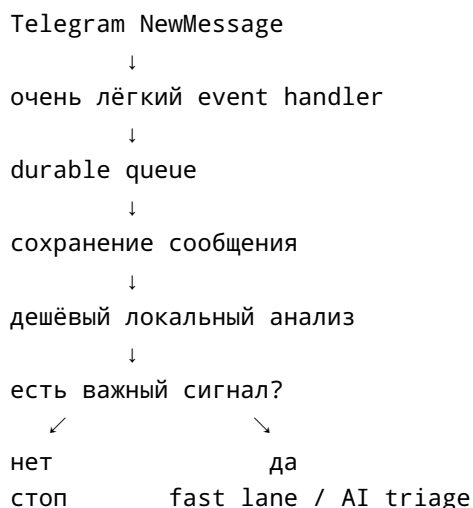
Итоговое заключение

Я разобрал загруженный архив `b2b ai-checker 2.zip` как систему целиком: backend, Telegram user-session через Telethon, клиентских ботов, Mini App, SQLite, фоновые worker-пулы, очередь задач, первоначальную загрузку истории, incremental ingestion, локальные сигналы, AI-triage, регулярный анализ, обязательства, уведомления, сотрудников, группы и API между фронтом и бэком.

Главный вывод: фундамент проекта уже хороший, но центральный механизм получения новых сообщений сейчас устроен не так, как должен быть устроен конечный продукт. Сейчас система фактически **раз в ~30 секунд просыпается, берёт каждый подключённый Telegram-аккаунт и проходит по всем выбранным чатам, проверяя, не появились ли сообщения.** Это не полный AI-анализ каждые 30 секунд, но всё равно лишняя Telegram-нагрузка, которая растёт пропорционально числу аккаунтов × числу чатов. Это видно по `services/backend/config.py:L52-L64`, `services/backend/scheduler/main.py:L31-L42`, `services/backend/scheduler/service.py:L197-L253` и `services/backend/telegram_sessions/incremental.py:L41-L100`.

Твоя новая идея — **не опрашивать каждый чат, а постоянно держать Telegram-сессию подключённой и реагировать на новые сообщения по событию** — архитектурно правильное для этого продукта. Более того, текущий Telethon-клиент прямо создаётся с `receive_updates=False`, то есть live updates сейчас намеренно выключены: `services/backend/telegram_sessions/gateway.py:L118-L126`. Telethon штатно поддерживает `events.NewMessage`, постоянное получение updates и `catch_up()` для событий, пропущенных во время отключения. ¹

Отдельно важно: **не надо удалять очереди, воркеры или AI pipeline.** Наоборот, event-driven Telegram должен стать только новым входом в уже существующую систему:



↓
проблема / обязательство
↓
админ / сотрудник / группа

То есть Telegram listener **не должен сам вызывать AI**. Он только принимает сообщение и быстро передаёт его в очередь. Именно это позволит событиям идти параллельно, но не конфликтовать друг с другом.

По моей оценке, **относительно твоего теперь уже уточнённого конечного продукта реализовано примерно 60% ± 8%**. Это не процент строк кода. Основа — multi-tenant, Telegram login, история, очередь, сигналы, AI, сотрудники, уведомления, Mini App, отчёты — уже существует. Но центральная часть конечной модели — **always-on мониторинг всех личных чатов + event-driven ingestion + опциональные группы как источники + полноценный invoice trigger** — ещё требует существенной переделки.

Что сейчас самое сильное и самое слабое

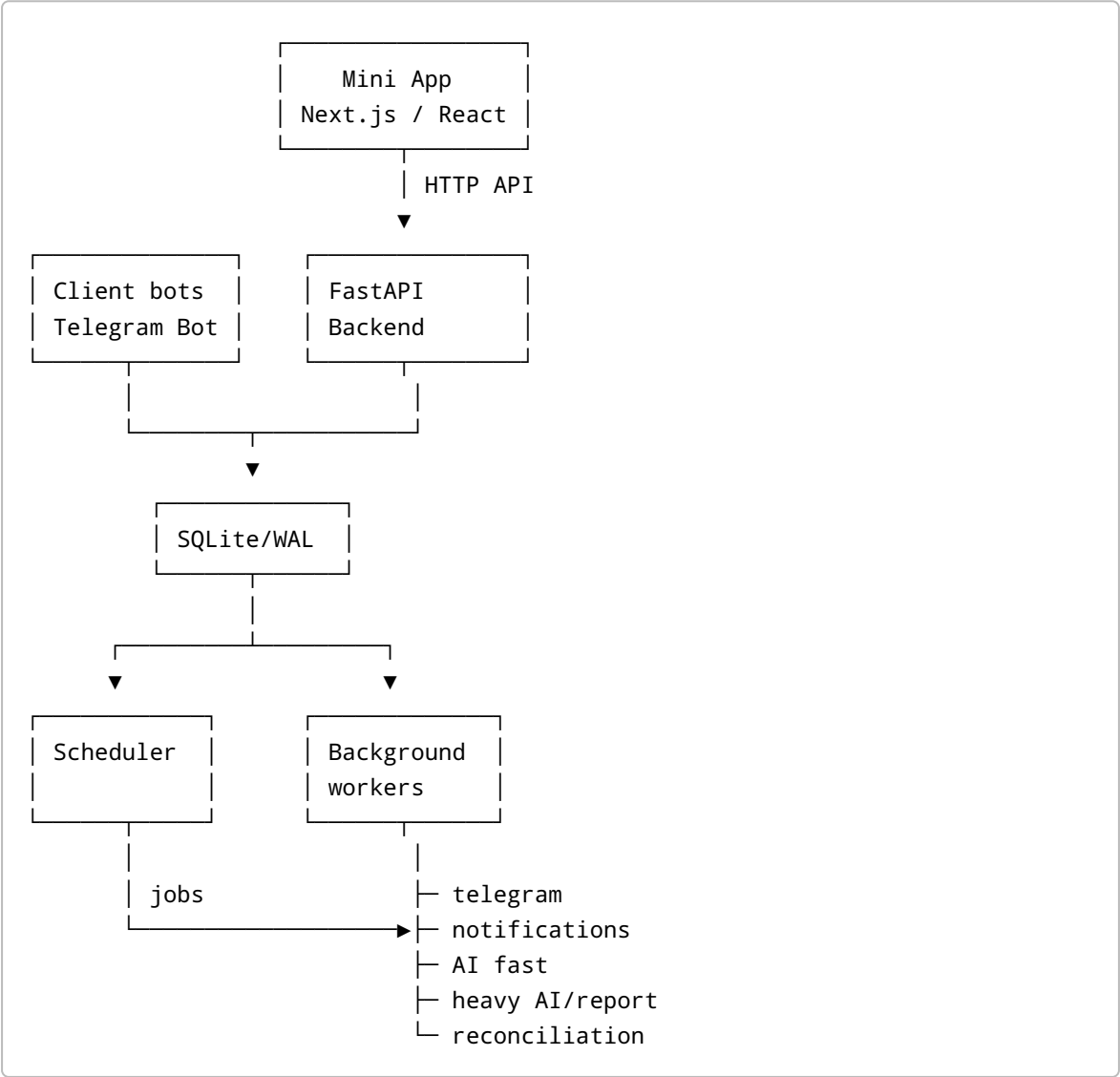
Область	Состояние	Моя оценка
Multi-tenant/backend foundation	● Хорошо	~85%
Telegram login и user sessions	● Хорошо	~80%
Первоначальная загрузка истории	● Работает	~75%
Live monitoring новых сообщений	● Нужна переделка	~25%
Локальный сигнал → AI triage	● Хорошая база	~80%
Fast lane критических событий	● Есть, но примитивный	~65%
Уведомление администратору	● Есть	~85%
Уведомление сотруднику	● Есть	~80%
Управление сотрудниками	● Есть	~80%
Группы как destination уведомлений	● Есть	~75%
Группы как источник мониторинга	● Почти нет	~20%
Добавление группы ссылкой	● Нет	~10%
Добавление Telegram-папки ссылкой	● Нет	~5%
Invoice-specific detection	● Есть основа	~45%
Regular reconciliation	● Есть	~80%
Regular AI reports	● Есть	~75%
Mini App	● Большая часть есть	~70%
Независимость frontend/backend	● В целом хорошая	~80%
Масштабирование БД/очереди	● MVP-уровень	~50%

Область	Состояние	Моя оценка
Production packaging/security	Требуется внимания	~45%

Как проект реально работает сейчас

В глобальном понимании проект уже состоит не из одного Telegram-бота, а из нескольких независимых процессов. `docker-compose.yml: L1-L64` поднимает backend API, owner bot, background worker, scheduler и client-bots. Все они при этом работают с одной SQLite-базой и общими каталогами `data / backups`.

Упрощённо архитектура сейчас такая:



Источник: `docker-compose.yml: L1-L64`, `services/backend/jobs/worker.py: L138-L228`, `services/backend/jobs/worker.py: L268-L325`.

Подключение Telegram

Пользователь через Mini App вводит телефон, код Telegram и при необходимости 2FA. Подключение может быть привязано к конкретному сотруднику через `assigned_employee_id`. API уже имеет tenant/permission checks, а Telegram session сохраняется отдельно от одноразового кода. Основа для нескольких аккаунтов одного проекта уже есть. Источник: `services/backend/api/client_router.py:L1135-L1202`, `services/backend/models.py:L342-L414`.

Но после подключения система сейчас заставляет пользователя пройти шаг **выбора Telegram-папки**. Frontend прямо содержит шаг `"folders"` и пишет:

«Ventrrix анализирует только выбранную рабочую папку».

Также есть отдельный checkbox личных диалогов. Источник: `app/mini-app/features/connections/connection-manager.tsx:L9-L20`, `L39-L53`, `L91-L119`.

Backend соответствует этому UX: `select_scope()` требует хотя бы одну папку и ставит `dialog.selected=True` только чатам из выбранных папок плюс, при отдельном consent, личным диалогам. Источник: `services/backend/telegram_sessions/service.py:L337-L394`.

Это полностью расходится с твоей новой моделью. Теперь именно этот слой надо менять: личные чаты становятся default source, а группы — дополнительным opt-in.

Первоначальная история

После выбора scope запускается initial sync. История загружается пачками по 100 сообщений, ограничена периодом истории и максимумом сообщений на чат. Сейчас default — 7 дней, допустимые значения 3/7/14/30. Источник: `services/backend/config.py:L33-L35`, `services/backend/models.py:L375-L377`, `services/backend/telegram_sessions/sync.py:L28-L38`.

Есть хороший механизм cursor:

```
Telegram chat
  ↓
batch 100
  ↓
store messages
  ↓
update offset
  ↓
если есть ещё → следующий job
```

Это правильная идея. Не нужно загружать огромную историю одним процессом.

Но после initial sync система берёт **все загруженные message IDs и создаёт отдельный `signal.local_scan` job на каждое сообщение:** `services/backend/telegram_sessions/sync.py:L351-L377`.

Для небольшого MVP это нормально. Если же сделать твоё новое правило «сразу все личные чаты», это станет первым серьёзным источником queue amplification.

Например, если первоначально загрузились:

```
200 личных чатов × 100 сообщений = 20 000 сообщений
```

то только стартовая обработка потенциально создаст около **20 000 отдельных local-scan jobs**. Это не означает 20 000 AI-запросов, но означает 20 000 строк и операций очереди.

Для новой версии лучше initial history обрабатывать **по batch/диалогу**, а не job-per-message.

Новые сообщения сейчас

Вот здесь находится основной архитектурный недостаток.

Scheduler работает циклом:

```
while True:
    await scheduler.tick()
    await asyncio.sleep(...)
```

с default poll interval 30 секунд. `services/backend/scheduler/main.py:L31-L42`.

На каждом operational scheduling cycle для **каждого активного TelegramConnection** создаётся:

```
telegram.fetch_updates
```

Источник: `services/backend/scheduler/service.py:L197-L240`.

После этого job получает список всех:

```
TelegramDialog.selected == True
TelegramDialog.excluded == False
```

и делает цикл по каждому диалогу. Для каждого выполняется `fetch_new_messages(... after_id=cursor.last_message_id)`. Источник: `services/backend/telegram_sessions/incremental.py:L41-L79`.

Telethon внутри делает:

```
client.iter_messages(
    dialog_id,
    min_id=after_id,
    reverse=True,
```

```
    limit=limit  
)
```

Источник: `services/backend/telegram_sessions/gateway.py:L302-L344`.

Поэтому текущая модель фактически:

```
каждые ~30 секунд  
↓  
каждый Telegram account  
↓  
каждый выбранный chat  
↓  
Telegram: "есть что-нибудь после message_id X?"
```

Именно эту часть я бы **убрал из основного realtime path**.

Что происходит после получения сообщения

Вот эта часть, наоборот, уже довольно хорошо построена.

Новое сообщение сохраняется, затем запускается дешёвый локальный detector. Он умеет видеть:

- вопросы о цене;
- договор/КП;
- оплату/счёт/invoice;
- документы;
- вложения;
- жалобы;
- коммерческий интерес;
- следующий шаг;
- обещания сотрудника;
- повторное сообщение клиента;
- клиента, ожидающего ответа.

Источник: `services/backend/intelligence/local_signals.py:L12-L45`, `L89-L208`.

Особенно правильная идея в том, что **далеко не каждое Telegram-сообщение автоматически идёт в AI**. Сначала дешёвые deterministic rules. Потом только сигналы выше threshold попадают в AI triage. Это именно тот подход, который нужен для экономии мощности и денег.

Источник: `services/backend/intelligence/signals.py:L70-L171`.

AI получает не всю историю, а ограниченный контекст последних сообщений плюс состояние диалога и открытые обязательства. По default context — 8 сообщений. Источник: `services/backend/config.py:L69-L69`, `services/backend/intelligence/ai_triage.py:L148-L209`.

Дальше AI решает:

```
criticality
requires_employee_notification
requires_manager_notification
deadline
needs_deep_analysis
```

И только при необходимости запускает тяжёлый deep analysis. Источник: `services/backend/intelligence/ai_triage.py:L57-L146`.

Это одна из наиболее удачных частей архитектуры.

Где сейчас будут тратиться мощности и возникать конфликты

Главный bottleneck — опрос всех диалогов

Проблема polling не столько в самом scheduler, сколько в следующей формуле.

Пусть:

```
A = подключённых Telegram-аккаунтов
D = мониторящихся диалогов на аккаунт
```

При интервале 30 секунд система теоретически делает примерно:

$$2 \times A \times D$$

проверок диалогов в минуту, даже когда никто ничего не написал.

Это вывод непосредственно из текущего цикла

```
scheduler → telegram.fetch_updates → for dialog in dialogs → iter_messages :
services/backend/scheduler/service.py:L197-L240 , services/backend/
telegram_sessions/incremental.py:L49-L91 .
```

Например, **не benchmark, а иллюстрация архитектурной сложности:**

```
20 аккаунтов
× 100 отслеживаемых чатов
× 2 проверки в минуту
≈ 4 000 проверок чатов в минуту
```

В event-driven модели при отсутствии новых сообщений работа стремится почти к нулю: Telethon уже имеет механизм update events, включая `NewMessage`; `catch_up()` предназначен для обработки пропущенных updates после регистрации handlers. ¹

Поэтому твоя мысль здесь правильная: **Telegram сам должен сказать Ventrix, что пришло новое сообщение.**

Сейчас live updates вообще отключены

Самая показательная строка проекта:

```
receive_updates=False
```

в `TelethonGateway.client()`. Источник: `services/backend/telegram_sessions/gateway.py:L118-L126`.

То есть сделать event-driven не означает писать какой-то самодельный WebSocket. Нужно перевести уже используемый Telethon в его стандартный режим receiving updates и зарегистрировать event handlers. Telethon поддерживает `add_event_handler(..., events.NewMessage)`, работу до disconnect и `catch_up()` после временного offline. ²

Внутри incremental ingestion есть лишние SQL-запросы на каждое сообщение

Сейчас `_store_batch()` для каждого нового сообщения:

1. делает SELECT — существует ли оно;
2. делает SELECT последних 10 сообщений;
3. INSERT;
4. `flush()`;
5. выполняет local signal scan;
6. повторяет всё для следующего сообщения.

Источник: `services/backend/telegram_sessions/incremental.py:L162-L226`.

Это классический потенциальный N×query участок. Он не страшен при нескольких сообщениях в секунду, но при backfill или большом burst станет заметным.

Причём initial sync сделан лучше: там existing IDs запрашиваются пачкой. Источник: `services/backend/telegram_sessions/sync.py:L125-L169`.

Я бы унифицировал два пути:

```
event/batch
  ↓
  одним SELECT получить existing IDs
  ↓
  bulk insert
  ↓
  один раз получить необходимый context
  ↓
  локальный анализ новых сообщений
```



```
↓  
commit
```

Повторный `has_more` job сейчас снова account-wide

В incremental-коде, если конкретный dialog имеет больше сообщений, создаётся новый job с idempotency key, в котором есть `dialog.id`, однако payload остаётся `{}`. Новый job снова запускает `fetch_updates()` и снова проходит весь список selected dialogs. Источник: `services/backend/telegram_sessions/incremental.py:L79-L91`.

Это не correctness bug — cursor и idempotency защищают от повторной вставки, — но это **лишняя работа**, особенно при большом backlog.

В event-driven варианте этот код почти исчезнет из realtime path; backlog reconciliation можно сделать per-dialog.

Пулы worker'ов разделены хорошо, но общий tenant limit создаёт скрытый конфликт

Это важный нюанс.

У тебя уже есть отдельные worker pools:

```
notification  
telegram / sync  
ai / ai_fast  
heavy / analysis / report  
general / reconciliation
```

Источник: `services/backend/jobs/worker.py:L268-L325`.

Это хорошо: тяжёлый report сам по себе не использует telegram worker.

Но queue дополнительно имеет:

```
max_active_tenant_jobs = 2
```

и при claim любого job проверяет общее число running jobs этого tenant. Если уже запущено два job этого проекта, следующий не берётся, независимо от категории. Источник: `services/backend/config.py:L36-L49`, `services/backend/jobs/queue.py:L175-L225`.

Отсюда возможна ситуация:

```
Tenant A:  
  
AI job #1          RUNNING
```

```
AI job #2          RUNNING
-----
tenant jobs = 2/2

приходит:
PO notification    WAIT
или
Telegram ingest    WAIT
```

То есть **отдельные worker pools существуют, но tenant-wide cap всё равно может связать их между собой.**

Это как раз противоречит твоему требованию: критическое новое сообщение не должно конфликтовать с уже идущим анализом.

Я бы разделил лимиты так:

```
realtime ingest    reserved capacity
notification       reserved capacity
AI fast            bounded
AI heavy           bounded
reports           lowest priority
```

И не позволял бы тяжёлым AI jobs исчерпывать capacity для `notification` и `telegram.ingest`.

SQLite пока подходит для MVP, но является потолком записи

Проект сознательно работает на SQLite и включает WAL:

```
journal_mode=WAL
synchronous=NORMAL
busy_timeout
```

Также есть process-local semaphore, сериализующий writes, и retry при `database is locked`.
Источник: `services/backend/database.py:L23-L34`, `L52-L70`, `L104-L129`.

Это довольно аккуратная SQLite-реализация.

Но semaphore **локальный для процесса**, а у тебя несколько процессов: backend, bots, worker, scheduler и т. д. Поэтому cross-process записи всё равно встречаются на уровне SQLite.

WAL позволяет reader'ам работать одновременно с writer'ом, но не превращает SQLite в multi-writer database; concurrent writers всё равно сериализуются. Официальная документация SQLite прямо описывает преимуществом WAL одновременную работу readers и writer, а также ограничение на общую файловую среду. ³

Поэтому:

сейчас переносить всё на Kafka + PostgreSQL + Redis не надо.

Но когда появятся десятки активно пишущих аккаунтов и постоянный event stream, именно SQLite write lock станет одним из первых инфраструктурных потолков.

Regular analysis — не то же самое, что polling каждые 30 секунд

Здесь важное уточнение.

Сейчас существуют три разных механизма:

```
~30 секунд → fetch новых Telegram сообщений
~1 час      → reconciliation обязательств/проблем
по расписанию → полноценный AI report
```

Источник: `services/backend/config.py:L52-L64`, `services/backend/scheduler/service.py:L142-L195`, `L197-L253`.

Hourly reconciliation **не перечитывает Telegram целиком**. Он работает по уже сохранённым `Commitment`, `OperationalProblem` и `TelegramMessage`, проверяет, выполнено ли обещание, просрочено ли оно и решена ли проблема. Источник: `services/backend/intelligence/reconciliation.py:L34-L100`.

Эту часть я бы **оставил**.

A scheduled AI report тоже преимущественно работает по локальной базе, не делает Telegram polling заново. Он создаёт analysis run, строит batches из сохранённых `TelegramMessage` и отправляет их в AI. Источник: `services/backend/analysis/service.py:L87-L160`.

Но здесь есть другой cost bottleneck: preprocessing для каждого selected dialog сначала загружает **все сообщения окна истории из БД**, а уже потом сжимает их до ограниченного контекста. Источник: `services/backend/analysis/preprocessing.py:L129-L172`; compact context — `L65-L114`.

Для большого проекта лучше:

```
не "каждый день заново AI для всех чатов",
а
"каждый день AI только для changed / meaningful dialogs
+ накопленное DialogState
+ открытые Problems
+ Commitments
+ агрегаты"
```

Иначе после масштабирования не Telegram polling, а **daily AI-per-dialog** станет главным денежным cost driver.

Полная история может взорвать очередь

После initial sync текущая реализация создаёт по одному `signal.local_scan` на каждый исторический message. `services/backend/telegram_sessions/sync.py:L351-L377`.

При переходе на «все личные диалоги» я бы изменил это **до включения default-all-personal**.

Правильнее:

```
telegram.history_batch
    ↓
сохранили 100 сообщений
    ↓
signal.scan_batch(message_ids=[...])
```

То есть один queue job на 50–200 сообщений, а не 100 отдельных jobs.

Mini App authentication тоже имеет будущий O(N) bottleneck

Backend правильно проверяет Telegram Mini App init data через HMAC. Но при каждом authenticated request он сейчас загружает **все активные BotInstance**, по очереди decrypt'ит их token и проверяет initData каждым token, пока не найдёт подходящий. Источник: `services/backend/api/client_router.py:L390-L478`.

При:

```
5 client bots
```

это незаметно.

При:

```
500 tenant bots
```

это уже неприятная модель.

Лучше после первоначальной Telegram validation выпускать короткоживущую серверную session для Mini App либо иметь проверяемый route/bot hint → один bot token lookup. При этом tenant_id нельзя просто доверять frontend'у.

Какая архитектура должна заменить текущий polling

Я бы зафиксировал принцип:

Push first → durable queue → local filter → AI only when needed → periodic reconciliation only as страховка.

Отдельный Telegram listener

Добавить ещё один runtime:

```
telegram-listener
```

или переименовать его концептуально в:

```
telegram-session-runtime
```

Он должен владеть активными user sessions.

Для каждого подключённого аккаунта:

```
TelegramConnection A
    ↓
long-lived Telethon client
    ↓
events.NewMessage
events.MessageEdited
    ↓
queue
```

Telethon штатно предоставляет `NewMessage` для новых текстовых и media-сообщений, а `MessageEdited` — для изменения уже существующего сообщения. ⁴

Это дополнительно исправит ещё один недостаток текущего polling: запрос `min_id > last_message_id` хорошо находит **новые ID**, но не предназначен для обнаружения изменения текста уже существующего сообщения. Поэтому edited invoice/amount/deadline сейчас может пройти мимо realtime detection.

Event handler должен быть почти пустым

Плохой вариант:

```
@client.on(events.NewMessage)
async def handler(event):
    save_to_db()
    query_history()
    call_ai()
    send_notifications()
```

Это создаст новую проблему вместо старой.

Правильный вариант:

```
@client.on(events.NewMessage)
async def handler(event):
    event_payload = normalize(event)

    await queue.enqueue(
        "telegram.ingest_event",
        event_payload,
        idempotency_key=...
    )
```

То есть handler:

```
не делает AI
не скачивает PDF
не анализирует 30 сообщений
не строит отчёт
не ждёт notification
```

Он должен освободиться максимально быстро.

Один владелец Telegram session

Здесь есть тонкость именно текущего проекта.

Сейчас `TelethonGateway.connected_client()` использует **per-session asyncio.Lock** и сериализует работу с одним Telegram client. Источник: `services/backend/telegram_sessions/gateway.py:L109-L175`.

Если просто взять этот context manager и внутри выполнить:

```
await client.run_until_disconnected()
```

то listener будет держать lock практически вечно, а history sync / health / другие Telegram RPC начнут ждать его.

Поэтому я бы не «добавил `NewMessage` в существующий worker» в лоб.

Лучше сделать:

```
Telegram Session Actor
|
├─ постоянно получает updates
├─ команда fetch history
├─ команда resolve group
```

```
└─ команда catalog refresh
└─ health/reconnect
```

Для каждого account один actor/client является владельцем session.

Внутри:

```
incoming updates → queue
RPC commands      → serialized through actor
```

Так ты гарантируешь:

одна session не используется конфликтующими процессами одновременно, но разные аккаунты прекрасно работают параллельно.

Catch-up вместо постоянного polling

После restart listener должен:

```
register handlers
→ connect
→ catch_up()
→ receive live updates
```

Telethon прямо рекомендует вызывать `catch_up()` после регистрации handlers, чтобы загруженные пропущенные updates тоже прошли через обработчики. ²

При этом я **не удалял бы reconciliation fetch полностью**.

Нужна страховка:

live event stream	постоянно
catch_up	после reconnect
light reconciliation	например, раз в 10–30 минут
full session health	периодически

Но reconciliation должен спрашивать:

«Есть ли gap после нашего cursor?»

а не:

«Давайте каждые 30 секунд зайдём во все 500 чатов».

То есть polling перестаёт быть основным ingestion mechanism и становится recovery mechanism.

Порядок сообщений внутри одного диалога

Ты правильно подчеркнул «без конфликтов с другими процессами, которые идут параллельно».

Правильное правило:

разные dialogs → параллельно

один dialog → последовательно по Telegram message_id

Например:

Chat A:

101 → 102 → 103

Chat B:

55 → 56

А и В могут идти параллельно.

Но 103 не должен изменить DialogState раньше 102.

Это важно для логики:

клиент написал вопрос

сотрудник ответил

клиент повторил вопрос

обещание выполнено/не выполнено

Нужен либо `dialog_id` partition key в очереди, либо лёгкий per-dialog lock/actor.

База уже имеет хорошую основу для idempotency

Есть уникальность:

(connection_id, telegram_dialog_id)

(dialog_id, telegram_message_id)

(connection_id, dialog_id) для incremental cursor

Источник: `services/backend/models.py:L433-L471`, `L549-L599`.

Поэтому ситуация:

NewMessage пришёл live

+

то же сообщение попало в catch_up


```
+  
reconciliation тоже его нашёл
```

не должна превращаться в три TelegramMessage.

Это хороший фундамент.

Я бы дополнительно закрепил event idempotency:

```
new:  
connection_id:peer_id:message_id:new  
  
edit:  
connection_id:peer_id:message_id:edited_at
```

Разделить приоритеты очереди

Целевая схема:

Lane	Что туда идёт	Приоритет
realtime	Telegram NewMessage	высокий
critical	critical fast lane	максимальный
notification	админ/сотрудник	максимальный
ai_fast	AI triage	высокий
reconciliation	сроки/обещания	средний
historical	initial history	низкий
ai_heavy	deep analysis	низкий
reports	регулярные отчёты	самый низкий

И главное правило:

```
Report никогда не должен блокировать NewMessage.  
Deep AI никогда не должен блокировать P0 notification.  
Initial history никогда не должен блокировать live traffic.
```

Текущие отдельные pool'ы уже позволяют сделать это без полного переписывания worker subsystem. Нужно в основном переработать admission limits в `SQLiteJobQueue`.

Как должны работать личные чаты, группы, инвойсы и уведомления

Личные чаты — по умолчанию все

Я бы полностью убрал обязательную настройку:

```
"Выберите рабочую папку"
```

После успешного Telegram login новая последовательность должна быть:

```
Telegram подключён
↓
получить dialog catalog
↓
BCE personal dialogs = monitor by default
↓
history window = 7 дней default
↓
historical bootstrap
↓
live listener active
```

Текущий default 7 дней уже есть в модели и UI, поэтому его удобно сохранить. Источник: `services/backend/models.py:L372-L381`, `app/mini-app/features/connections/connection-manager.tsx:L39-L43`.

Модель:

```
personal chat → enabled by default
group         → disabled by default
channel       → disabled by default
```

Я бы не использовал `selected` как основную бизнес-модель в будущем.

Вместо этого:

```
connection.source_policy.personal_default = true

MonitoredSource:
  connection_id
  telegram_peer_id
  source_type
```

```
enabled
added_via
```

или хотя бы:

```
monitoring_mode:
    default
    include
    exclude
```

Так появление нового personal dialog не требует повторного catalog selection: listener видит новый peer, понимает, что это private chat, и начинает мониторить автоматически.

Нужно решить судьбу автоматического исключения «личных» личных чатов

Сейчас initial analysis классифицирует personal dialogs и при выраженных бытовых признаках делает:

```
selected = False
excluded = True
```

Источник: `services/backend/telegram_sessions/sync.py:L278-L316`.

То есть даже если пользователь разрешил личные диалоги, система позже может исключить разговор как `personal_contact`.

Если твоя новая бизнес-логика буквально:

«отслеживаем все личные чаты»

то это поведение надо изменить.

Оптимальный компромисс:

```
получать события из всех personal chats
↓
дешёвый local relevance gate
↓
не бизнес-чат?
↓
не отправлять в AI / не создавать problems
```

То есть **monitoring source** ≠ **AI processing source**.

Это позволит не пропустить клиента, который внезапно написал по работе, но не тратить AI на бытовое общение.

Группы должны стать отдельным дополнительным источником

Сейчас сущность `GroupIntegration` существует, но её смысл в основном:

куда можно отправлять group reminders

В API есть создание группы по уже известному `telegram_chat_id`, title и notification settings. Источник: `services/backend/api/client_router.py:L1491-L1573`.

Mini App фактически только показывает существующие группы и позволяет переключить notifications:

```
api.groups()  
api.updateGroup(...)
```

Источник: `app/mini-app/features/sections/section-views.tsx:L51-L56`.

Причём `VentrixClientApi` имеет `updateGroup()`, но даже не имеет frontend-метода `createGroup()`, хотя backend POST существует. Источник: `app/mini-app/api/client.ts:L76-L78`, `L161-L172`.

То есть **твоя новая функция “добавить группу в мониторинг” сейчас фактически отсутствует.**

Как должен выглядеть раздел «Источники»

Я бы объединил текущие `Connections` и часть `Groups` в понятную модель:

ИСТОЧНИКИ

Аккаунт: @ivan_sales

- Подключён

Личные диалоги

- ✓ Все новые и существующие

История: 7 дней

Дополнительные группы

- ✓ TechCorp × Ventrix
- ✓ Client Project Alpha
- Team Internal

[+ Добавить группу]

Добавить:

[Вставить ссылку на группу]
[Вставить ссылку на Telegram-папку]

При этом employee/admin connection management можно оставить отдельно.

Ссылку на группу можно обрабатывать user-session

Telegram API имеет `messages.checkChatInvite`: user account может проверить invite link и получить информацию о чате до импорта; документация отмечает, что метод предназначен для user accounts. ⁵

Поэтому flow может быть:

```
пользователь вставил t.me/... / +invite
↓
parse link
↓
connected Telegram user-session
↓
check invite / resolve username
↓
показать:
"Project X, 18 участников"
↓
[Включить мониторинг]
```

Если аккаунт уже находится в группе — просто создаём monitored source.

Если не находится — вступление должно быть отдельным явным действием.

Ссылка на Telegram-папку тоже технически возможна

Telegram поддерживает shared chat folder links. API позволяет сначала выполнить `chatlists.checkChatlistInvite`, получить содержимое folder deep link, а затем при необходимости импортировать выбранные peers через `chatlists.joinChatlistInvite`. Telegram отдельно указывает, что пользователь может выбрать, какие группы/каналы из folder invite присоединить. ⁶

Значит твоя идея:

«передал ссылку на папку — Ventrrix увидел группы внутри»

реализуема.

Но я **не делал бы автоматический join ко всем группам сразу**, потому что импорт Telegram folder может реально менять список чатов connected user account. Telegram API рассматривает проверку invite и импорт как разные операции. ⁷

Правильный UX:

```
Вставлена ссылка на folder
↓
Preview:
[✓] Client A
[✓] Client B
[ ] Internal Team
[ ] News Channel
↓
"2 группы будут добавлены в мониторинг"
↓
Подтверждение
```

Invoice detection уже частично есть, но пока слишком примитивен

Это важный момент: слово `invoice` в проекте **уже есть**.

`PAYMENT_RE` содержит:

```
счёт
счет
invoice
оплат...
реквизит...
ндс
предоплат...
```

Источник: `services/backend/intelligence/local_signals.py:L15-L20`.

Есть и generic признак attachment:

```
"attachment": bool(message.attachments_json)
```

Источник: `services/backend/intelligence/local_signals.py:L119-L136`.

Но между ними пока нет полноценного invoice detector.

Например файл:

```
Invoice_2389_August.pdf
```

с пустой подписью сейчас будет в первую очередь:

```
document_request / attachment
```

а слово `invoice` в имени файла отдельно через `PAYMENT_RE` не проверяется.

Telethon gateway уже сохраняет:

```
attachment.kind
attachment.name
attachment.size
attachment.mime_type
```

Источник: `services/backend/telegram_sessions/gateway.py:L319-L340`.

То есть все данные для хорошего cheap invoice trigger почти уже есть.

Я бы сделал отдельный local detector:

```
invoice_score =
    text/caption keyword
+ filename keyword
+ financial document MIME/ext
+ amount/currency
+ incoming direction
+ external/customer sender
+ commercial context
```

Например:

"Счёт на оплату" + PDF	→ ВЫСОКИЙ
invoice_4382.pdf	→ ВЫСОКИЙ
"Реквизиты во вложении" + PDF	→ ВЫСОКИЙ
"оплатите 250 000 ₽"	→ ВЫСОКИЙ
random_photo.jpg	→ НОЛЬ

Fast lane уже есть — его надо расширить, а не переписывать

Текущий `critical_fast_lane_rules` позволяет:

```
signal_types
contains_any
contains_all
criticality
enabled
```

Источник: `services/backend/intelligence/signals.py:L97-L105`, `L173-L192`.

При совпадении fast lane система даже умеет отправить **предварительное critical notification ещё до AI**, а потом AI может подтвердить его или отменить/исправить уведомление. Источник: `services/backend/intelligence/signals.py:L140-L171`, `services/backend/intelligence/notifications.py:L298-L383`.

Это очень удачный механизм.

Я бы расширил rule format до:

```
{
  "enabled": true,
  "signal_types": ["invoice_received"],
  "contains_any": ["счёт", "invoice", "оплата"],
  "attachment_name_any": ["invoice", "счет", "счёт"],
  "mime_types": ["application/pdf"],
  "directions": ["incoming"],
  "source_types": ["personal", "group"],
  "requires_amount": false,
  "sender_roles": ["customer", "external"],
  "criticality": 90
}
```

Тогда critical invoice может пройти путь:

```
NewMessage
  ↓ ~milliseconds/local
invoice detector
  ↓
fast lane
  ↓
P0 provisional notification
  ↓
parallel AI triage
  ↓
confirm / correct
```

Это значительно лучше, чем отправлять **каждый новый invoice-like message** в тяжёлый AI прежде чем уведомить человека.

Не надо скачивать каждый PDF

Сейчас gateway сохраняет только metadata attachment, а не содержимое файла. Для realtime trigger это даже плюс.

Целевая схема:


```
любой PDF
↓
filename/text/mime local detector
↓
неинтересный → не скачивать

похож на invoice
↓
document-analysis job
↓
скачать документ
↓
извлечь сумму / контрагента / срок
```

Так документы не станут новым bandwidth/CPU bottleneck.

Уведомление администратору уже реализовано

Notification policy проверяет manager threshold, а orchestrator отправляет manager destination на:

```
tenant.owner_telegram_user_id
```

Источник: `services/backend/intelligence/notifications.py:L38-L82`, `L159-L221`.

То есть схема:

```
важный signal
→ manager notification
→ бот владельца проекта
```

уже есть.

Уведомления сотрудникам тоже уже реализованы

Есть:

```
Employee.telegram_user_id
notifications_enabled
criticality_threshold
quiet_hours
```

и центральная policy проверяет попор tenant + employee threshold + AI decision + quiet hours.

Источник: `services/backend/models.py:L1094-L1121`, `services/backend/intelligence/notifications.py:L38-L82`.

`SignalService` умеет сопоставлять `sender/connection` с сотрудником. Источник: `services/backend/intelligence/signals.py:L218-L251`.

Это означает, что твоё требование:

«если админ подключил сотрудников — уведомлять и нужного сотрудника»

уже реализовано на хорошем уровне.

Но для рабочих групп нужно исправить понятие «клиент»

Это один из наиболее важных скрытых багов будущей group monitoring модели.

`LocalSignalEngine` сейчас считает:

```
external_sender = not message.outgoing
```

и многие правила также используют `not message.outgoing`. Источник: `services/backend/intelligence/local_signals.py:L112-L143`, `L187-L207`.

В личном диалоге connected work account это в целом разумно:

```
outgoing = сотрудник  
incoming = клиент
```

Но в group chat:

```
connected account = Иван  
  
сообщение от сотрудницы Анны  
→ incoming относительно аккаунта Ивана  
→ outgoing=False
```

Хотя Анна может быть зарегистрированным Employee.

Сейчас Employee mapping определяется **после local detector**, в `SignalService`. Источник: `services/backend/intelligence/signals.py:L79-L89`, `L218-L251`.

Поэтому при добавлении групп есть риск, что:

```
сообщение другого сотрудника
```

будет локально воспринято как:

```
external/customer message
```

и создаст false-positive `waiting_customer` или `customer_question`.

До запуска group monitoring local engine должен получать:

```
sender_role:  
  account_owner  
  employee  
  customer  
  external  
  unknown
```

а не ориентироваться только на `outgoing`.

Независимость фронта от бэка, качество реализации и production-риски

Frontend и backend в целом разделены правильно

Mini App не ходит напрямую в БД и почти не содержит backend business logic.

Есть центральный `VentrixClientApi`, который получает:

```
NEXT_PUBLIC_API_BASE_URL  
Telegram initData
```

и общается с `/api/v1/client/...`. Источник: `app/mini-app/api/client.ts:L25-L45`, `app/mini-app/hooks/use-mini-app-session.ts:L9-L49`.

UI маршрутизирует разделы:

```
dashboard  
problems  
commitments  
statistics  
reports  
employees  
connections  
groups  
settings
```

а компоненты получают API abstraction. Источник: `app/mini-app/mini-app-root.tsx:L34-L49`.

То есть frontend можно переработать под новый “Источники” UX, почти не затрагивая intelligence pipeline. Это хороший уровень разделения.

Но есть risk: TypeScript types/API contracts описываются вручную, а не генерируются из OpenAPI schema. При постре backend это создаёт риск frontend/backend drift.

Сейчас folder-based модель слишком глубоко зашита в UX

Не только UI, но и onboarding state использует:

```
scope_selection
folders
selected_folder
personal_dialogs_consent
```

Источник: `app/mini-app/features/connections/connection-manager.tsx:L9-L20`, `L39-L53`, `L91-L119`; `services/backend/telegram_sessions/service.py:L337-L394`.

Поэтому изменение «все личные диалоги автоматически» — это не одна галочка. Потребуется миграция:

```
DB model
connection service
API schemas
onboarding flow
Mini App connection wizard
incremental eligibility
scheduled analysis eligibility
initial sync
tests
```

Regular AI analysis можно сделать намного дешевле

Сейчас `AnalysisBatchBuilder` для каждого selected dialog делает запрос всех TelegramMessage за history window и складывает prepared data в память, после чего context compacted до 32 000 символов. Источник: `services/backend/analysis/preprocessing.py:L65-L114`, `L129-L224`.

Переход к event-driven открывает возможность постепенно поддерживать:

```
DialogState
rolling summary
latest meaningful message
open commitments
open problems
```

```
last meaningful version
last analyzed version
```

У тебя `DialogState` и `meaningful_version` уже есть, то есть начало этой архитектуры существует. Источник: `services/backend/intelligence/signals.py:L282-L329`.

Поэтому scheduled report в будущем может делать:

```
было meaningful_version=18
последний report видел version=18
→ чат не изменился → AI не нужен

meaningful_version=23
→ проанализировать delta
```

Это может дать гораздо больший выигрыш по AI-cost, чем любое микропрофилирование Python.

Multi-account duplicate detection нужно усилить

Текущий signal fingerprint:

```
tenant_id : internal message.id : signal_type
```

Источник: `services/backend/intelligence/signals.py:L90-L96`.

Это отлично защищает один message row от повторного local scan.

Но если один и тот же Telegram group наблюдается **через два рабочих аккаунта одного tenant**, Telegram-сообщение сохранится как разные внутренние `TelegramMessage` в разных connections, а fingerprints будут различаться.

При будущей group monitoring модели возможны duplicate signals:

```
manager account увидел сообщение
employee account увидел то же сообщение
→ два internal messages
→ два signals
```

Стоит добавить cross-connection dedup для групп:

```
tenant_id
canonical_peer_id
telegram_message_id
signal_type
```

или content/time fingerprint.

Архив проекта сам по себе сейчас небезопасен для распространения

Это не архитектура runtime, но это важный production finding.

В переданном архиве находятся:

```
.env
data/app.db
backups/
.git/
.venv/
node_modules/
.next/
```

Сам проект при этом в `.gitignore` явно исключает `.env`, DB, runtime data, backups, virtualenv, `node_modules` и `.next`. Источник: `.gitignore:L3-L55`.

Более того, уже есть специальный `scripts/export_source.py`, который **намеренно отказывается экспортировать** `.env` и **runtime paths** и запускает secret scan перед созданием source archive. Источник: `scripts/export_source.py:L29-L70`.

То есть этот конкретный ZIP был создан не через безопасный exporter.

Размеры распакованной структуры также показывают проблему:

```
node_modules  ≈ 921 MB
.venv         ≈ 227 MB
.next         ≈ 105 MB
.pyru_cache   ≈ 27 MB
data          ≈ 5.5 MB
backups       ≈ 3.2 MB
```

А всего внутри оказалось около **39,5 тыс. файлов**, хотя исходного Python-кода — порядка 103 файлов, TypeScript/TSX — порядка 22.

Для следующих архивов надо использовать уже существующий:

```
python -m scripts.export_source
```

и **никогда не передавать production** `.env`, **рабочую DB** и **backup** вместе с source archive.

Я не раскрываю содержимое `.env` или БД в этом отчёте.

Автотесты присутствуют, но этот архив не является воспроизводимой test environment

В проекте есть полноценный `tests/backend` и `pyproject.toml` содержит `aiogram` и `telethon`.

Однако встроенный `.venv` оказался привязан символической ссылкой к локальному macOS Python:

```
/Library/Frameworks/Python.framework/Versions/3.13/bin/python3.13
```

поэтому в моей Linux sandbox его Python executable использовать нельзя. Системный Python при test collection не имел `aiogram` / `telethon`, и pytest остановился на девяти import errors. Это **не означает, что девять тестов проекта падают**; означает, что bundled virtualenv непереносим между машинами.

Frontend lint прошёл до стадии Next build, после чего Next попытался скачать платформенный Linux SWC package, но sandbox не имела доступа к npm registry. Поэтому честно заявить «все тесты зелёные» по этому архиву я не могу.

Правильная CI-проверка должна каждый раз делать fresh environment:

```
clean checkout
→ python -m venv
→ pip install -e '.[dev]'
→ alembic upgrade head
→ pytest
→ npm ci
→ npm test
→ source export / secret scan
```

а не полагаться на присланные `.venv` / `node_modules`.

Чеклист фактической готовности

Функция	Статус	Что уже есть / чего не хватает
Multi-tenant проекты	✓	Tenant boundary проходит через основные модели/API
Owner bot	✓	Отдельный runtime
Client bots per tenant	✓	BotInstance и отдельный client-bots runtime
Telegram user-session login	✓	Телефон, code, 2FA
Привязка Telegram account к employee	✓	<code>assigned_employee_id</code>

Функция	Статус	Что уже есть / чего не хватает
Шифрование Telegram session	✓	EncryptedSecret abstraction
Загрузка catalog Telegram dialogs	✓	folders + dialogs
Историческая загрузка сообщений	✓	cursor, batch, history_days
Ограничение истории 3/7/14/30	✓	DB + API + UI
Все personal chats по умолчанию	✗	Сейчас нужны folder/consent/selected
Автоматическое подключение нового personal chat	✗	Нужен live listener + default policy
Live <code>NewMessage</code> monitoring	✗	<code>receive_updates=False</code>
Обработка edited messages	✗	Polling по min_id их не решает
Recovery после disconnect	●	Retry есть; event <code>catch_up</code> ещё нет
Polling каждые 30 сек	⚠	Есть, надо вывести из realtime path
Durable queue	✓	Idempotency, retries, leases, heartbeat
Fairness между tenants	✓	<code>TenantQueueState</code>
Отдельные worker pools	✓	telegram / AI / notification / heavy
Защита realtime от heavy AI	●	Пулы есть, но общий tenant cap может блокировать
Per-dialog ordering	✗/●	Явного partition/actor порядка нет
SQLite WAL	✓	Настроен аккуратно
Масштабирование multi-writer	●	SQLite станет потолком
Batch dedup сообщений	✓	Unique remote message constraint
Cross-account dedup одного group message	✗	Нужен canonical peer dedup
Local cheap signal detector	✓	Хорошая база
Цена/КП/договор	✓	Regex + signals
Счёт/invoice text detection	✓	<code>PAYMENT_RE</code>
Invoice filename detection	✗	Attachment name пока не участвует
MIME-based invoice conditions	✗	MIME хранится, rule не использует
Amount/currency contextual invoice trigger	●	Amount logic есть в report preprocessing, не fast lane
Fast lane	✓	Provisional notification before AI
Fast lane <code>contains_any/all</code>	✓	Реализовано

Функция	Статус	Что уже есть / чего не хватает
Fast lane filename/MIME/sender/source conditions	✗	Нужен richer rule DSL
AI triage	✓	Separate fast AI path
Ограниченный контекст AI	✓	Последние сообщения + state
AI token budgets	✓	Soft/hard controls существуют
Deep analysis only when needed	✓	<code>needs_deep_analysis</code>
Employee commitments	✓	Promise → Commitment
Deadline reconciliation	✓	Просроченные обязательства
Auto problem resolution	✓/●	Reconciliation + verifier
Hourly reconciliation	✓	Отдельный механизм
Daily scheduled reports	✓	Analysis schedules
Incremental report optimisation	●	DialogState есть, но полный reuse ещё недостаточен
Notification manager/admin	✓	Owner Telegram destination
Notification employee	✓	Threshold + quiet hours
Notification group	✓	GroupIntegration
Notification dedup/cooldown	✓	Есть
Correction provisional alert after AI	✓	Есть
Employees Mini App	✓	Добавление/редактирование
Permissions	✓	Есть membership/permissions
Groups Mini App list	✓	Есть
Добавить group через Mini App	●	Backend POST есть, frontend flow отсутствует
Group как ingestion source	✗	Сейчас в основном notification integration
Добавить group invite link	✗	Нужно реализовать
Добавить public <code>t.me/name</code>	✗	Нужно resolve flow
Добавить shared folder link	✗	Нужно реализовать preview/import
Sender roles внутри группы	✗	<code>not_outgoing</code> недостаточно
Раздел «Источники»	✗	Нужен новый UX
Frontend/backend API separation	✓	Хорошая
Generated frontend API types	✗	Сейчас ручные contracts

Функция	Статус	Что уже есть / чего не хватает
Mini App HMAC auth	✓	Проверяется
Auth масштабирование на много bots	●	Сейчас перебор всех active bots
Production source export	✓	Скрипт уже есть
Фактически переданный безопасный archive	✗	ZIP содержит runtime/secrets-related files
CI reproducible clean build	●	Код/тесты есть, текущий ZIP не позволяет подтвердить

Моя итоговая оценка прогресса

С учётом **твоего нового целевого поведения**, а не старого folder-based сценария:

Блок	Вес	Готовность
Platform/backend foundation	15%	85%
Telegram login + history	10%	75%
Event-driven live monitoring	20%	25%
Signals + AI intelligence	15%	80%
Employees + notifications	10%	80%
Groups as monitored sources	10%	25%
Mini App	10%	65–70%
Production/scalability/security	10%	45–50%

Итого: около 60% готовности относительно конечной версии, которую ты сейчас описываешь.

При этом это хороший тип «60%»: не ситуация, где надо выбросить проект и начать заново. Наоборот, уже готовы самые неприятные инфраструктурные части:

```
tenants
sessions
encryption
database models
jobs
retry
AI triage
signals
problems
commitments
employees
notifications
```

Переделывать в основном нужно **входной Telegram ingestion + source policy + group ingestion UX**, а не всю систему.

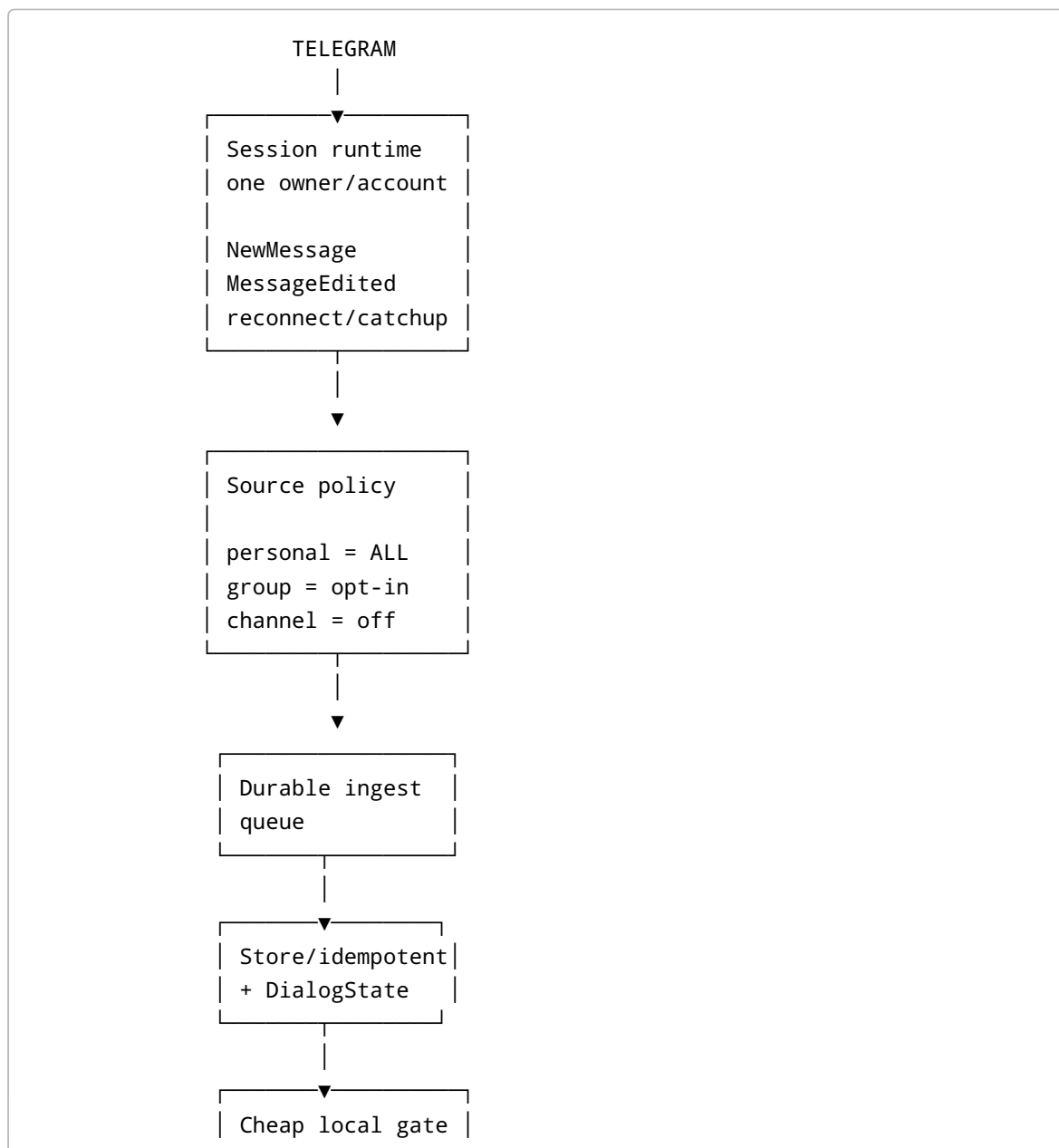
Приоритетная архитектурная дорожная карта

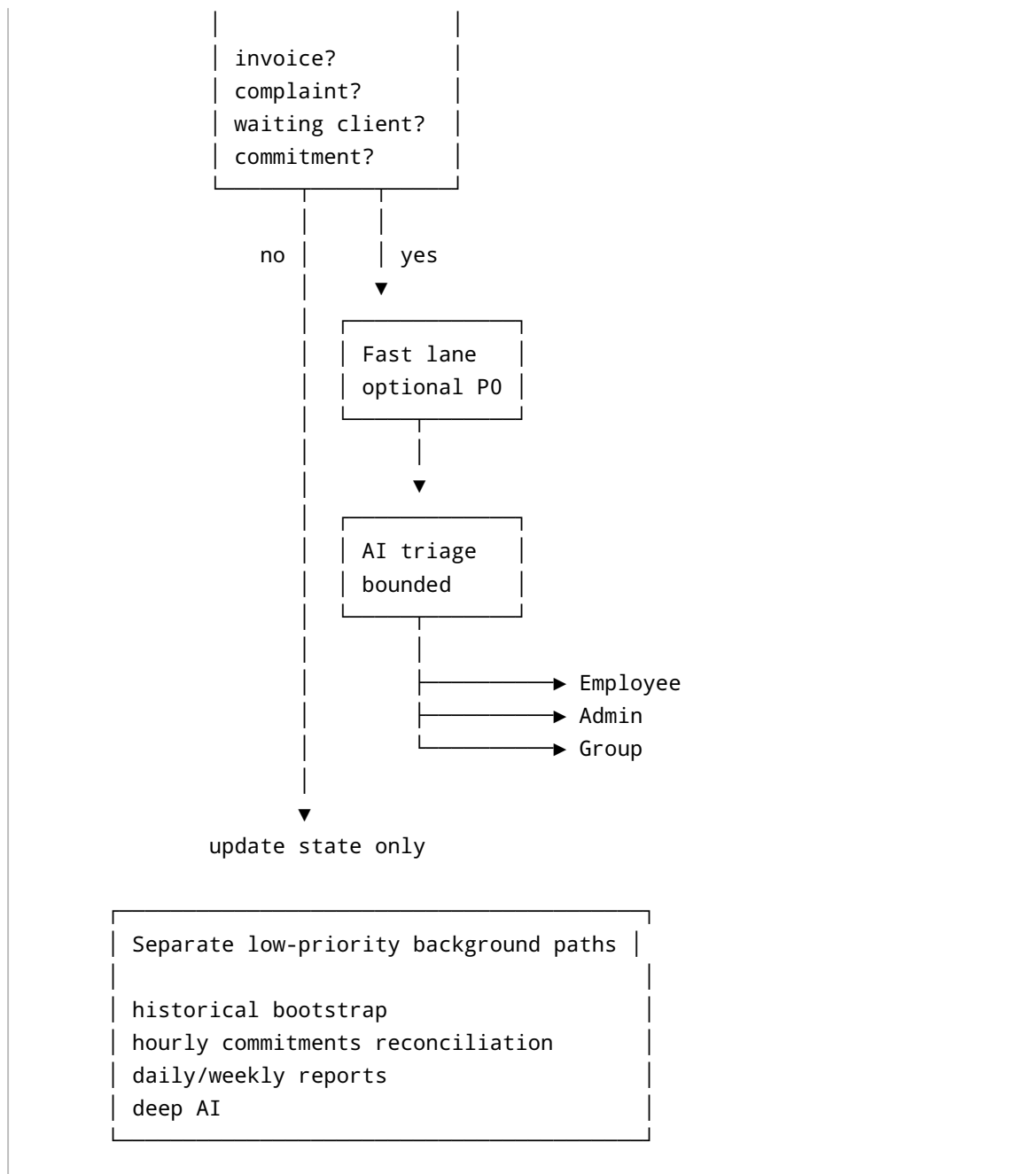
Я бы не начинал сейчас с косметических функций Mini App. Самый правильный порядок такой:

Приоритет	Изменение	Почему первым
P0	Убрать <code>.env</code> /DB/backups/build artifacts из рабочих архивов	Защита секретов и данных
P0	Создать long-lived Telegram session runtime	Основа новой архитектуры
P0	Включить <code>receive_updates</code> + <code>NewMessage</code>	Убирает основной polling
P0	Сделать <code>telegram.ingest_event</code>	Изолирует Telegram от analysis
P0	Idempotent NewMessage/catch-up ingestion	Не будет дублей
P0	Reserved realtime/notification queue capacity	AI не блокирует критические события
P0	Перевести personal chats в default-enabled	Твоя новая core product logic
P0	Переделать initial scan с job-per-message на batch	Иначе default-all-personal раздует очередь
P1	Добавить MessageEdited	Исправляет изменение invoice/сроков
P1	Rich invoice local detector	Быстрый critical trigger
P1	Filename/MIME/amount conditions	Уменьшает false positives
P1	Rich fast-lane rule schema	Настраиваемые критические события
P1	Новый Mini App «Источники»	Нормальный UX новой модели
P1	Group source by link	Основной paid add-on
P1	Folder-link preview	Массовое подключение клиентских групп
P1	Sender role awareness in groups	Без этого будут ложные «клиенты»
P1	Cross-account group dedup	Не дублировать сигналы
P2	Incremental DialogState/report versioning	Сильно снизит AI cost

Приоритет	Изменение	Почему первым
P2	AI only for changed dialogs	Масштабирование регулярного анализа
P2	Оптимизировать SQL ingestion	Убирает per-message N×queries
P2	Mini App authenticated server session/ cache	Убирает O(number of bots) auth
P2	Observability: queue lag / event lag / AI latency	Появятся реальные SLO
P3	PostgreSQL	Когда SQLite write contention подтвердится метриками

Конечная архитектура, к которой я бы вёл Ventrix, выглядит так:





Именно такая модель решает сразу несколько твоих требований: **никакого постоянного обхода каждого чата, автоматический мониторинг всех личных диалогов, дополнительные группы только по желанию, мгновенная реакция на новый invoice/критическую фразу, параллельность без блокировки realtime тяжёлым анализом, уведомление админа и сотрудника, а Mini App остаётся панелью управления, а не местом бизнес-логики.**

Возможность event-driven updates и recovery через Telethon соответствует штатной модели библиотеки, а shared-folder/group links можно проверять через user-session Telegram API. ⁸

¹ ² ⁸ TelegramClient — Telethon 1.44.0 documentation

https://docs.telethon.dev/en/stable/modules/client.html?utm_source=chatgpt.com

³ Write-Ahead Logging

https://www.sqlite.org/wal.html?utm_source=chatgpt.com

4 Events Reference — Telethon 1.44.0 documentation

https://docs.telethon.dev/en/stable/quick-references/events-reference.html?utm_source=chatgpt.com

5 `messages.checkChatInvite`

https://core.telegram.org/method/messages.checkChatInvite?utm_source=chatgpt.com

6 7 Dialog folders

https://core.telegram.org/api/folders?utm_source=chatgpt.com